Hoopla

User Manual

Interactive Front-End Modelling Software
for Galaxy-Scale Strong Gravitational Lensing

Repository (front-end):
https://gitee.com/long-jianchao/Hoopla-Preprocessing_input_pics

Repository (back-end):
https://gitee.com/long-jianchao/hoopla-backend-service

Version 1.0

Contents

1 Introduction **2**

2 Installation and Environment **2**
2.1 System Architecture and Platform Requirements . . . 3
2.2 Front-End Installation . . . 3
2.3 Back-End Environment and Dependencies . . . 4
2.4 Front-End/Back-End Communication . . . 4

3 Data Preparation **4**
3.1 Required Input Files . . . 4
3.2 Supported File Formats . . . 5

4 User Interface Layout **5**
4.1 Interactive Canvas Area (Left Panel) . . . 6
4.2 Optimisation Diagnostic Area (Centre and Right Panels) . . . 7
4.3 Workflow Control Bar (Bottom) . . . 7

5 Interactive Modelling Workflow **8**
5.1 Step 1 — Data Upload . . . 8
5.2 Step 2 — Arc Extraction (Optional) . . . 8
5.3 Step 3 — Interactive Initial Model Construction . . . 10
5.4 Step 4 — Front-End Numerical Optimisation . . . 10
5.5 Step 5 — Model Export and Comparison . . . 11
5.6 Step 6 — Back-End MCMC Sampling (Optional) . . . 12

6 Noise Handling and PSF Convolution **13**
6.1 Weight Map Convention . . . 14
6.2 PSF Convolution . . . 14

7 Physical Models **14**
7.1 Lens Mass Model: Singular Isothermal Ellipsoid (SIE) . . . 14
7.2 Source Photometric Model: Elliptical Sérsic Profile . . . 15

8 Frequently Asked Questions **16**

9 Limitations and Usage Recommendations **17**
9.1 Model Assumption Constraints . . . 17
9.2 Arc-Extraction Performance Limitations . . . 17
9.3 Limitations of Front-End Parameter Estimates . . . 17
9.4 Operational Recommendations . . . 18

References **18**

1 Introduction

Hoopla is a browser-based, interactive front-end modelling application designed for galaxy-scale strong gravitational lensing research. It provides an intuitive, high-efficiency interface that bridges observational imaging data and physical lens modelling, with a focus on the human-in-the-loop phase of the overall modelling pipeline. Through interactive canvas-based operations and real-time visual feedback, Hoopla enables users to construct and inspect initial parametric lens models prior to automated or high-fidelity refinement.

Hoopla follows a modular, pipeline-centric design philosophy and is intended to operate as the central hub of a complete modelling workflow:

- **Upstream integration.** Hoopla accepts structured input from deep-learning-based image segmentation modules. In the current implementation, *GGSL-UNet* [2] is used to automatically extract arc structures from gravitational lens images, supplying cleaned source images as input to the interactive modelling stage.
- **Downstream integration.** Lens mass models and source photometric models derived during interactive modelling are passed directly to back-end refinement frameworks. In the current implementation, *TinyLensGPU* [1] is used for JAX-accelerated high-precision parameter sampling (MCMC/nested sampling). The interface design preserves extensibility: alternative segmentation methods or back-end solvers may be substituted without modifying the front-end codebase.

The system imposes no prerequisite familiarity with the underlying statistical inference machinery. Novice users can explore the lens geometry through canvas-level dragging and real-time image-plane rendering; advanced users retain explicit control over model parameters, weight-map conventions, and PSF handling.

It is important to emphasise that Hoopla is not a standalone lens solver and does not produce final, statistically rigorous parameter constraints. Its design objective is to provide a well-defined, controllable, and extensible front-end interaction layer positioned between raw observational data and back-end high-fidelity modelling, thereby improving the efficiency and interpretability of the overall strong-lensing analysis pipeline.

Quick-start path. First-time users are advised to proceed as follows:

1. Complete installation and environment setup (Section 2).
2. Prepare the lens image, weight map, and PSF file (Section 3).
3. Upload the files, adjust the model ellipses interactively, and execute the front-end optimisation (Section 5).
4. If higher-fidelity constraints are required, invoke the back-end MCMC module via the `MCMC` button.

The physical background of the adopted mass and source models is detailed in Section 7.

2 Installation and Environment

Hoopla follows a front-end/back-end separated architecture. The front-end handles interactive modelling and visualisation; the back-end coordinates network communication, arc-extraction computation, and fine-modelling dispatching.

2.1 System Architecture and Platform Requirements

Table 1 summarises the two runtime components and their respective roles.

Table 1: Hoopla system components and platform requirements.

Component	Technology	Role and requirements
Front-end	Node.js / Web	Renders interactive canvases, parameter panels, and optimisation diagnostics. Runs in any modern browser; cross-platform (Windows, macOS, Linux).
Back-end	Python / FastAPI	Coordinates HTTP communication between the browser and compute modules; dispatches arc-extraction (GGSL-UNet) and fine-modelling (TinyLensGPU) tasks. Requires Python $\geq$ 3.8.

2.2 Front-End Installation

The front-end build toolchain is based on **Node.js** (LTS release recommended, $\geq$ v14). Verify the installation with:

```
node -v
npm -v
```

Proceed with the following steps to install and launch the application.

Step 1 — Clone the repository

```
git clone https://gitee.com/long-jianchao/Hoopla-Preprocessing_input_pics.git
cd Hoopla-Preprocessing_input_pics
```

Step 2 — Install npm dependencies

```
npm install
```

Step 3 — Launch the application

Three launch modes are available; select the one appropriate to the deployment context.

a) **Python HTTP server (recommended for local use)**

```
npm start
# Access via browser at http://localhost:8000
```

b) **Webpack development server (supports hot-reload)**

```
npm run dev
# Browser opens automatically
```

c) **Production build for static deployment**

```
npm run build
# Build artefacts are written to ./dist/
# Deploy the dist/ directory to any static web server
```

2.3 Back-End Environment and Dependencies

The back-end service is built with **FastAPI** and can be cloned from:

https://gitee.com/long-jianchao/hoopla-backend-service

The minimum Python version is 3.8. Two external computation modules are integrated.

TinyLensGPU. A JAX-accelerated galaxy-scale strong-lensing modelling framework developed by Cao et al. [1]. It provides the MCMC/nested-sampling back-end invoked by the `MCMC` button. Platform constraints are:

- **Linux** is the primary supported operating system.
- **Windows** users must install WSL 2 and configure an Ubuntu environment inside it.
- **macOS** (Apple Silicon) requires the `jax-metal` plugin to enable GPU acceleration; CPU-only execution is also supported.
- An **NVIDIA GPU** with CUDA 12.x is required for GPU-accelerated sampling. The CUDA installation must match the GPU model exactly.

Hoopla does not re-package the TinyLensGPU installation procedure. Users are directed to the official repository for full installation instructions:
https://github.com/caoxiaoyue/TinyLensGpu

GGSL-UNet. A deep convolutional segmentation model developed by Zhong et al. [2], trained to simultaneously extract the foreground lens galaxy, background source, and arc structure from galaxy-scale strong-lens images. The model produces segmented outputs at near-second latency. Source code and documentation are available at:
https://github.com/Fucheng-Zhong/GGSL-UNet

2.4 Front-End/Back-End Communication

The front-end and back-end communicate exclusively via HTTP. The front-end, running in the browser, transmits image data, model parameters, and user actions to the back-end FastAPI service; the back-end returns computation results (arc masks, optimisation progress, MCMC diagnostics) which are rendered in-browser. Both components can be deployed on the same host (local deployment) or on separate machines accessible over a local network.

3 Data Preparation

Before initiating an interactive modelling session, three categories of input file must be prepared. All files must be stored as two-dimensional numerical arrays in a supported format, and the three files must share an identical pixel grid, spatial extent, and coordinate convention.

3.1 Required Input Files

(1) Lens image. The observed or simulated image of the strong gravitational lens system, representing the surface-brightness distribution in the image plane. This file is mandatory. Hoopla does not perform upstream preprocessing; the supplied image is assumed to be in a state ready for scientific analysis, *i.e.* it should have undergone bias and dark subtraction, flat-field correction, sky-background subtraction, and cosmic-ray rejection prior to ingestion.

(2) Weight map. A two-dimensional map encoding the per-pixel photometric uncertainty. The weight map is distinct from an actual noise realisation extracted from the image: it must be a smooth, spatially varying description of the measurement standard deviation, free from Poisson, read-out, or any other stochastic fluctuations. Specifically:

- Each pixel value represents the 1σ standard deviation of the surface-brightness measurement at that position.
- The map encodes the spatial distribution of observational uncertainty, not actual noise fluctuations.
- The array must not contain negative values or NaN entries.

(3) Point Spread Function (PSF). A small two-dimensional kernel characterising the instrumental and atmospheric blurring introduced during the observation. The PSF array must satisfy the following conditions:

- The array is normalised to unit sum, *i.e.* $\sum_{i,j} \mathrm{PSF}_{ij} = 1$, to preserve total flux.
- The peak of the PSF is centred on the central pixel of the array.
- The PSF array dimensions are substantially smaller than those of the lens image.

3.2 Supported File Formats

Hoopla currently supports two standard astronomical data formats:

- **FITS** (Flexible Image Transport System)
- **HDF5** (Hierarchical Data Format version 5)

In both cases, each of the three input files must be stored as an independent two-dimensional array. Mixed-format combinations (*e.g.* lens image in FITS, weight map in HDF5) are permitted provided the pixel grids are consistent.

Once the three input files are prepared, Hoopla can be launched and the files uploaded; Section 5 describes the complete interactive modelling workflow.

4 User Interface Layout

The Hoopla interface is divided into three functional regions: the interactive canvas area (left), the optimisation diagnostic area (centre/right), and the workflow control bar (bottom). Figure 1 illustrates the overall layout.

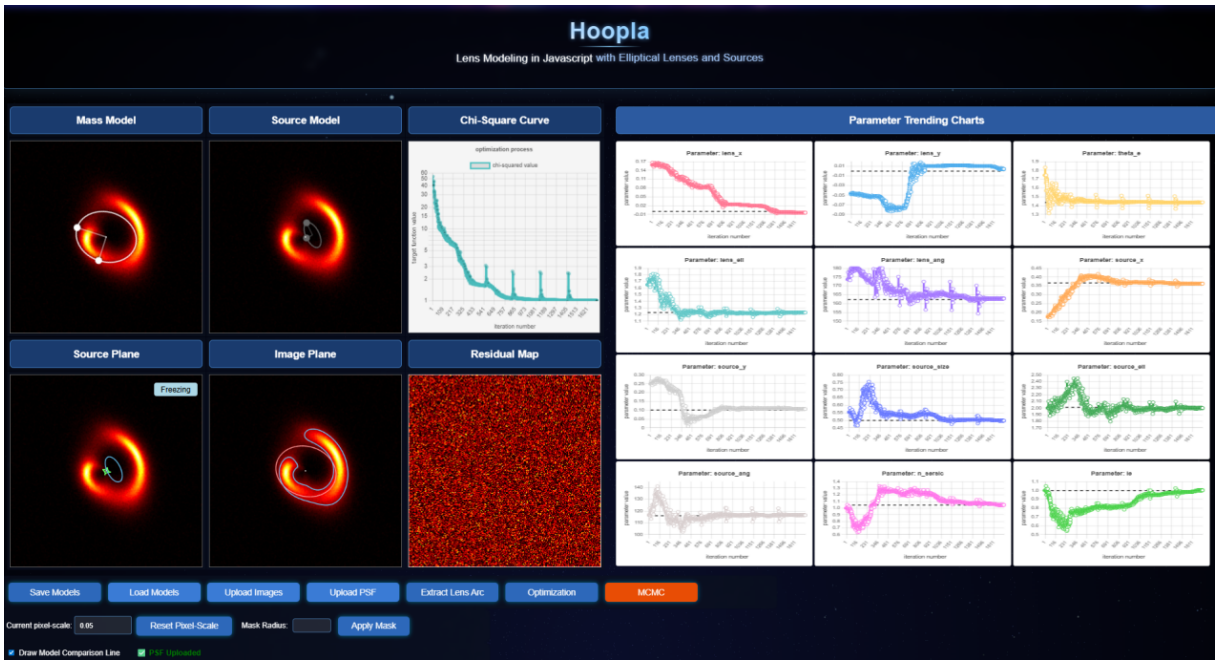

Figure 1: Hoopla main interface. Left: four interactive canvases. Centre and right: optimisation diagnostics. Bottom: workflow control bar.

4.1 Interactive Canvas Area (Left Panel)

The left panel contains four sub-canvases, each visualising a distinct component of the lens model.

(1) Mass Model canvas. Displays the geometric representation of the current lens mass distribution. The drawn ellipse encodes the model parameters — axis ratio q_L, position angle ϕ_L, and centroid (x_L, y_L) — in the image plane. The ellipse size serves as a visual scale reference for the Einstein radius θ_E and does not correspond to a physical boundary. Users interactively adjust the ellipse by click-and-drag. For the physical background of the underlying mass model, see Section 7.

(2) Source Model canvas. Displays the two-dimensional surface-brightness distribution of the background source in the source plane. The drawn ellipse represents the equal-intensity contour of a Sérsic profile, parameterised by the effective radius R_e, axis ratio q_S, and position angle ϕ_S. Users interact with this canvas in the same manner as the Mass Model canvas. For the underlying photometric model, see Section 7.

(3) Source Plane canvas. Renders the geometric outline of the source galaxy (blue curve) together with the caustic structure (green curve) in the source plane. Moving the cursor over this canvas repositions the source centroid; the corresponding lensed image geometry in the Image Plane canvas updates in real time. Matching the source position to the caustic geometry allows the user to reproduce the observed arc multiplicity and morphology.

(4) Image Plane canvas. Displays the lensed image contours (blue) predicted by the current mass–source model combination, together with the critical curves (red) mapped from the caustics via the lens equation. The morphology of the predicted arcs is determined jointly by the mass model, the source photometric model, and the relative source–lens geometry.

4.2 Optimisation Diagnostic Area (Centre and Right Panels)

χ^2 curve. Plots the value of the objective function χ^2 as a function of iteration number. Convergence is indicated by a stable plateau; persistent large-amplitude oscillations in the late optimisation phase indicate a failure to converge (see also Section 8).

Residual map. Displays the pixel-by-pixel difference between the observed image and the current model reconstruction. Systematic patterns in the residual map indicate unmodelled structures or biased parameter estimates; a noise-dominated residual map is consistent with an adequate fit.

Parameter evolution traces. A panel of 12 sub-plots showing the trajectory of each model parameter across optimisation iterations. These traces allow the user to identify whether parameters have entered a stable region, whether degeneracies are present, or whether a parameter is diverging to unphysical values.

4.3 Workflow Control Bar (Bottom)

Table 2 lists all controls in the bottom bar.

Table 2: Workflow control bar — button reference.

Control	Function
`Save Models`	Serialises the current model parameter set to a `.json` file and downloads it to the local file system.
`Load Models`	Uploads a previously saved `.json` parameter file and restores the corresponding model state.
`Upload Images`	Opens the *Image Settings* dialog for uploading the lens image, weight map, and PSF, and for specifying the pixel scale.
`Upload PSF`	Auxiliary shortcut for uploading the PSF file independently, provided to prevent accidental omission.
`Extract Lens Arc`	Sends the uploaded lens image to the back-end GGSL-UNet module for foreground subtraction and arc extraction; the processed image is returned and displayed on the canvas. A valid FITS or HDF5 file must be uploaded before invoking this function.
`Optimization`	Initiates front-end numerical optimisation of the mass and source model parameters using the Nelder–Mead simplex algorithm, minimising the objective function χ^2.
`MCMC`	Transmits the current model parameters as prior estimates to the back-end TinyLensGPU module and triggers an MCMC sampling run.
`Reset Pixel-Scale`	Resets the pixel scale to its default value.
`Apply Mask`	Applies a circular mask of radius specified in the adjacent *Mask Radius* field, restricting the fitting region to the central science area and excluding background noise or contaminating sources.

Status indicators. Table 3 lists the status indicators displayed in the control bar.

Table 3: Status indicators in the workflow control bar.

Indicator	Description
`Interactive`	The Source Plane canvas is in interactive mode; cursor movement controls the source centroid position. Click to toggle to `Freezing`.
`Freezing`	The Source Plane canvas is frozen; cursor movement does not affect the source position. This state is entered automatically upon optimisation completion. Click to return to `Interactive`.
`Draw Model Comparison Line`	When checked, a reference parameter trace is overlaid on the parameter evolution plots after a model file is loaded via `Load Models`, enabling comparison with reference values or results from other runs.
`PSF Not Uploaded` / `PSF Uploaded`	Indicates whether a valid PSF has been loaded. The status must read `PSF Uploaded` before executing `Optimization` or `MCMC`.

5 Interactive Modelling Workflow

5.1 Step 1 — Data Upload

Click `Upload Images` to open the *Image Settings* dialog. Upload the lens image, weight map, and PSF in sequence, then enter the pixel scale in arcseconds per pixel. Confirm by clicking `Confirm`. The upload can be aborted at any time via `Close`. The PSF status indicator in the control bar will update to `PSF Uploaded` upon successful ingestion.

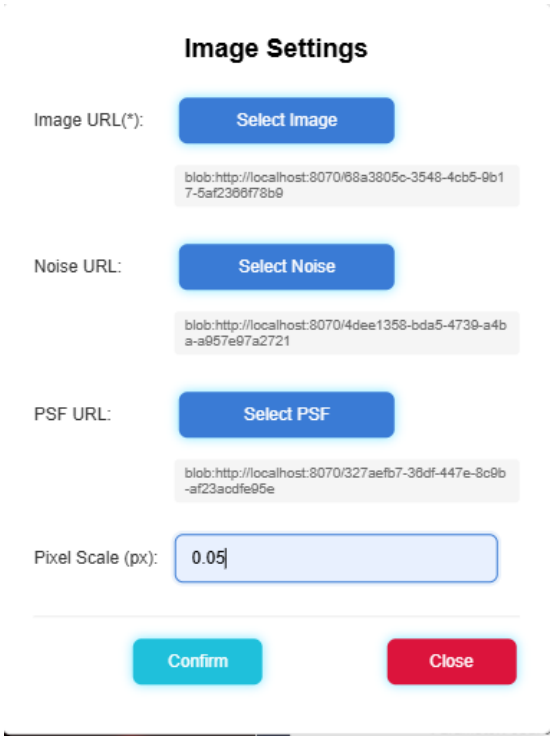

Figure 2: *Image Settings* dialog for uploading the lens image, weight map, and PSF, and for specifying the pixel scale.

5.2 Step 2 — Arc Extraction (Optional)

If the uploaded lens image retains the foreground lens galaxy light, click `Extract Lens Arc` to invoke the GGSL-UNet foreground subtraction pipeline. The processed image, with foreground

flux removed, is returned and displayed in the canvas area (Figures 4–5). If the user supplies a pre-subtracted image, this step may be skipped.

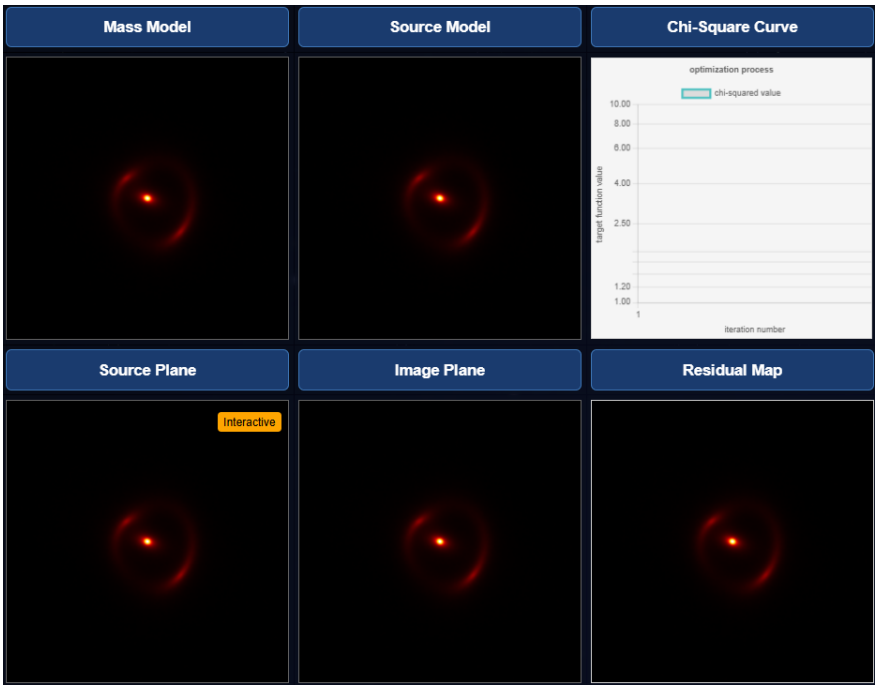

Figure 3: Original strong-lens image prior to foreground subtraction, containing the foreground lens galaxy flux. Click `Extract Lens Arc` to invoke the GGSL-UNet segmentation pipeline.

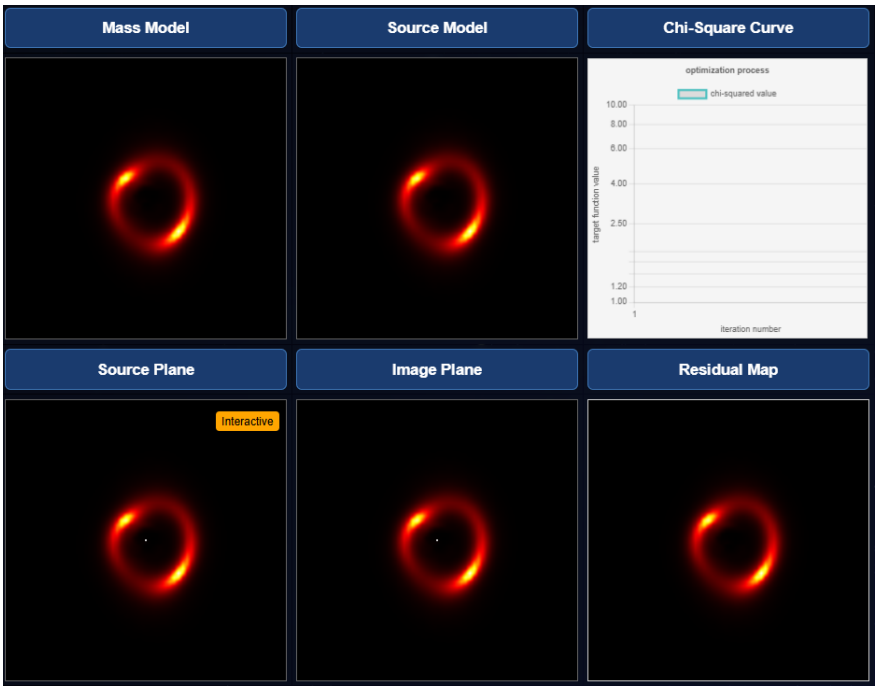

Figure 4: Arc extraction result (pass 1): output image after foreground galaxy suppression.

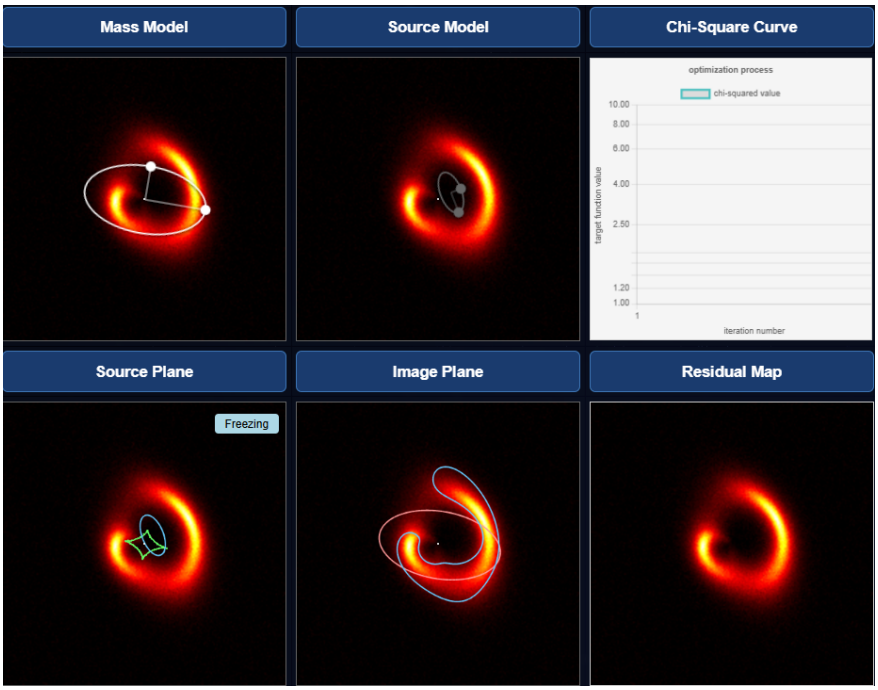

Figure 5: Arc extraction result (pass 2): lensed background source image ready for interactive modelling.

5.3 Step 3 — Interactive Initial Model Construction

On the **Mass Model** canvas, click and drag to define an ellipse encoding the initial estimates of θ_E, q_L, ϕ_L, and (x_L, y_L). Repeat on the **Source Model** canvas to define the initial Sérsic source ellipse $(R_e, q_S, \phi_S, x_S, y_S)$.

Navigate to the **Source Plane** canvas. While the canvas is in `Interactive` mode, cursor movement repositions the source centroid relative to the caustics. The Image Plane canvas updates in real time, rendering the predicted arc morphology. Adjust the source position until the predicted image-plane contours enclose the observed arc structure, indicating that a physically reasonable initial configuration has been reached.

5.4 Step 4 — Front-End Numerical Optimisation

Click `Optimization` to initiate the Nelder–Mead simplex optimisation. This derivative-free algorithm performs a bounded parameter-space search suited to the low-to-moderate dimensionality of the model parameter set (12 free parameters). During optimisation, the χ^2 curve, parameter evolution traces, and residual map update continuously. When the optimisation converges, the Source Plane canvas displays the `Freezing` indicator, signalling that the parameter estimates have stabilised (Figure 6).

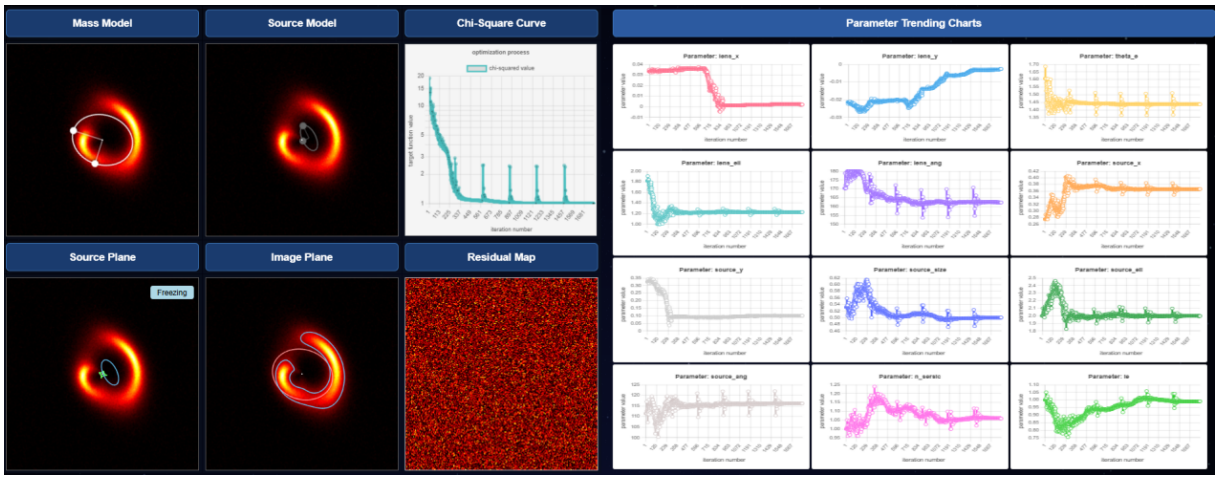

Figure 6: Optimisation diagnostics: χ^2 convergence curve (left) and parameter evolution traces (right), displayed during and after the Nelder–Mead optimisation run.

5.5 Step 5 — Model Export and Comparison

Click `Save Models` to export the current parameter set as a `.json` file (Figure 7). To compare results against a reference model (*e.g.* ground-truth simulated parameters or results from a prior run), click `Load Models` to upload a `.json` file; reference traces are overlaid on the parameter evolution plots if *Draw Model Comparison Line* is checked (Figure 8).


```
{
    "pixscale": 0.05,
    "components": [
        {
            "plane": "source",
            "size": 0.42662288598378056,
            "ell": 1.5474352860878346,
            "ang": -83.85322570800781,
            "x": 0.24464123536982008,
            "y": -0.2340311761259678,
            "n_sersic": 1.987828850577806,
            "Ie": 1.0
        },
        {
            "plane": "lens",
            "theta_e": 1.3826318560512056,
            "ell": 1.1933329558797547,
            "ang": 52.14274597167969,
            "x": 0.0,
            "y": 0.0
        }
    ]
}
```


Figure 7: Example of a model parameter `.json` file exported via `Save Models`.

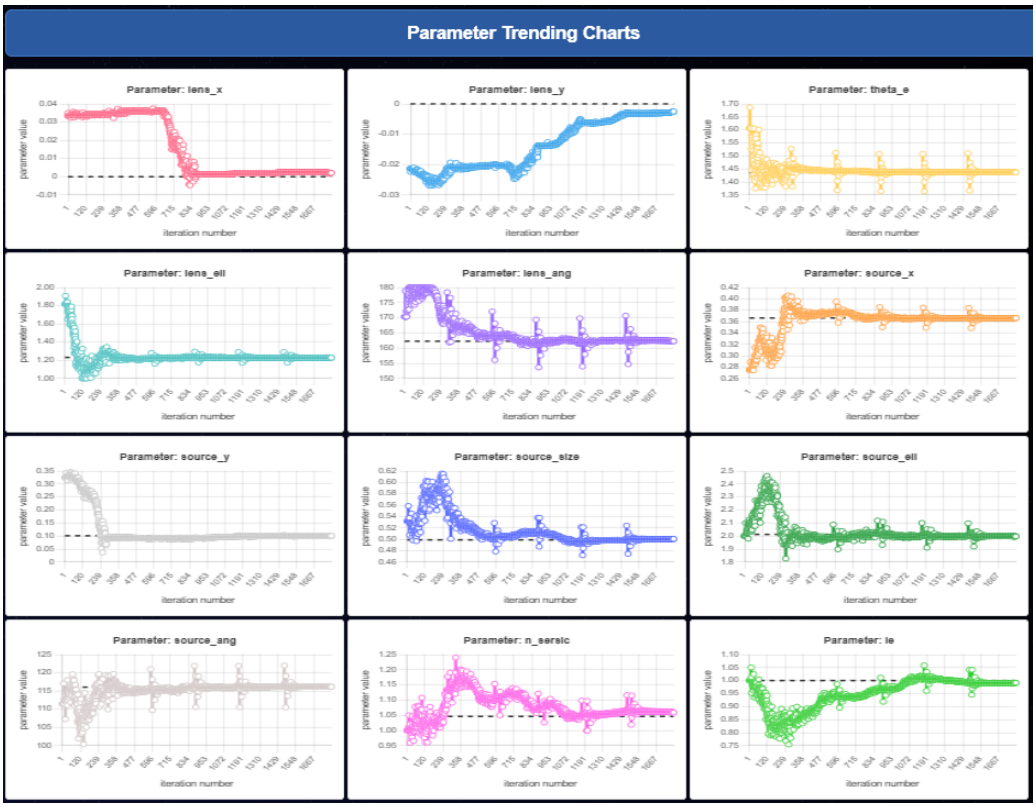

Figure 8: Parameter evolution plots with a reference model trace overlaid after loading a `.json` file via `Load Models`.

5.6 Step 6 — Back-End MCMC Sampling (Optional)

For statistically rigorous parameter inference, click `MCMC`. The current front-end model parameters are transmitted to TinyLensGPU as prior initialisation for the MCMC chain. A progress animation and live sampling diagnostics are rendered in the browser during sampling (Figures 9–10). The MCMC results should be regarded as the primary scientific output; the front-end optimisation result serves as the high-quality starting point for the sampler.

Figure 9: MCMC sampling progress animation displayed in-browser during back-end TinyLens-GPU execution.

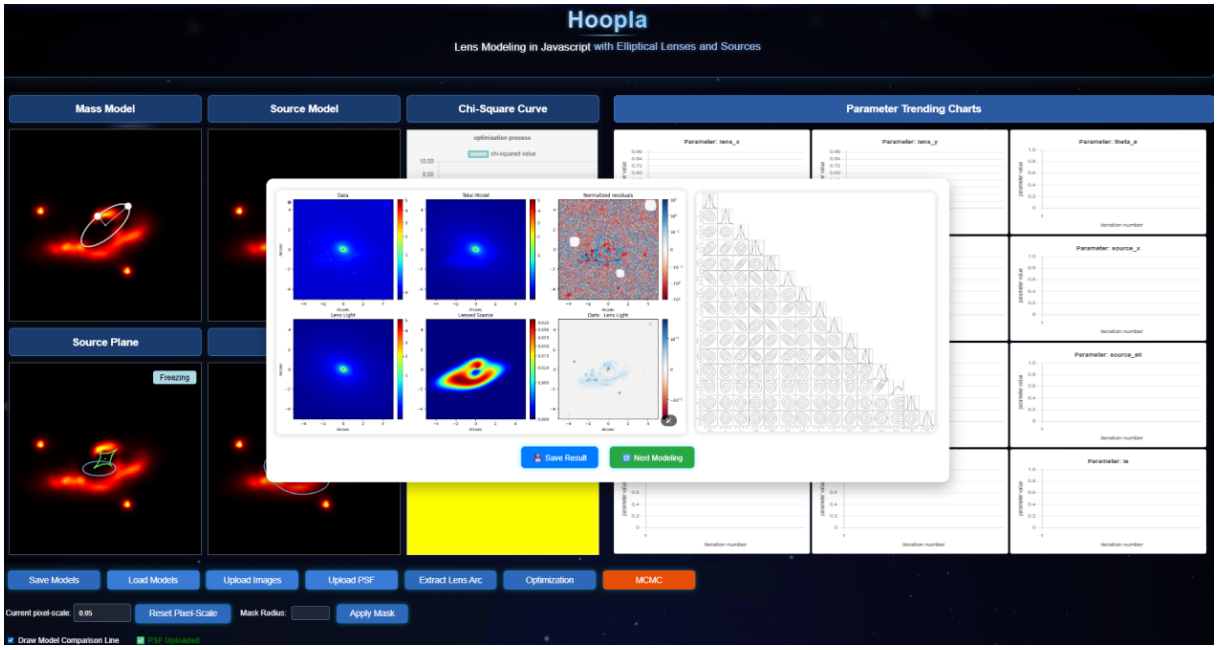

Figure 10: Live sampling diagnostic output streamed from the back-end to the browser during MCMC execution.

6 Noise Handling and PSF Convolution

Accurate characterisation of the noise properties and the instrumental PSF is essential for stable parameter optimisation and unbiased model reconstruction. Hoopla applies both quantities in a well-defined and consistent manner described below.

6.1 Weight Map Convention

Hoopla does not infer a noise model from the image data. The user-supplied weight map is used directly as the uncertainty description at every pixel. The weight map is interpreted as a smooth, spatially varying 1σ standard deviation map and must satisfy the following constraints:

- Each pixel value σ_{ij} represents the standard deviation of the surface-brightness measurement at pixel (i, j).
- The map encodes the spatial variation of observational uncertainty and must not contain any stochastic noise realisation (*i.e.* no Poisson or read-out noise fluctuations).
- All pixel values must be strictly positive; no NaN or zero entries are permitted.

The objective function minimised during optimisation is the standard weighted least-squares statistic:

$$\chi^2 = \sum_{i,j \in \mathcal{M}} \left(\frac{d_{ij} - m_{ij}}{\sigma_{ij}} \right)^2, \tag{1}$$

where d_{ij} is the observed surface brightness, m_{ij} is the model prediction at pixel (i, j), σ_{ij} is the weight-map value, and $\mathcal{M}$ denotes the set of unmasked pixels.

6.2 PSF Convolution

Prior to comparison with the observed image, the model image is convolved with the PSF using a standard discrete two-dimensional convolution:

$$m_{ij}^{\text{conv}} = \sum_{k,l} m_{i-k,\, j-l} \cdot \text{PSF}_{kl}, \tag{2}$$

where the PSF kernel is treated as spatially invariant across the field of view. The normalisation condition $\sum_{k,l} \text{PSF}_{kl} = 1$ ensures that total flux is conserved through the convolution. The PSF must be uploaded before any optimisation or MCMC invocation; a missing or incorrectly normalised PSF will lead to systematically biased model comparisons.

7 Physical Models

7.1 Lens Mass Model: Singular Isothermal Ellipsoid (SIE)

The lens mass distribution is parameterised by the Singular Isothermal Ellipsoid (SIE) model, which assumes the lens galaxy is dynamically isothermal with a three-dimensional mass-density profile

$$\rho(r) \propto r^{-2}, \tag{3}$$

producing an approximately flat rotation curve in projection. The SIE is the most widely adopted mass model for galaxy-scale strong-lensing analyses.

Elliptical radius

In the projected two-dimensional lens plane, the iso-convergence contours are elliptically symmetric. The elliptical radius is defined as

$$R = \sqrt{x'^2 + \frac{y'^2}{q_L^2}}, \tag{4}$$

where $q_L \in (0, 1]$ is the projected axis ratio of the mass distribution and (x', y') are coordinates in the lens principal-axis frame. The transformation from the image-plane coordinate system (x, y) is

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\phi_L & \sin\phi_L \\ -\sin\phi_L & \cos\phi_L \end{pmatrix} \begin{pmatrix} x - x_L \\ y - y_L \end{pmatrix}, \tag{5}$$

where (x_L, y_L) is the centroid of the mass distribution and ϕ_L is the position angle of the major axis measured from the image-plane horizontal axis.

Convergence profile

The dimensionless surface mass density (convergence) of the SIE model is

$$\kappa(R) = \frac{\Sigma(R)}{\Sigma_{\rm cr}} = \frac{\theta_E}{2R}, \tag{6}$$

where θ_E is the Einstein radius, which sets the characteristic angular scale of the lensing effect and directly determines the lensing magnification power. The critical surface mass density $\Sigma_{\rm cr}$ is

$$\Sigma_{\rm cr} = \frac{c^2}{4\pi G} \frac{D_s}{D_d D_{ds}}, \tag{7}$$

with D_d, D_s, and D_{ds} denoting the angular diameter distances from the observer to the lens, from the observer to the source, and from the lens to the source, respectively.

Relation to velocity dispersion

For the isothermal model, the Einstein radius and the line-of-sight velocity dispersion σ_v of the lens galaxy are related by

$$\theta_E = 4\pi \left(\frac{\sigma_v}{c}\right)^2 \frac{D_{ds}}{D_s}, \tag{8}$$

indicating that θ_E is a direct proxy for the overall gravitational potential strength of the lens.

Parameter set

Within the Hoopla framework, the SIE mass model is fully described by the five-parameter vector

$$\boldsymbol{\pi}_L = \{\theta_E,\ x_L,\ y_L,\ q_L,\ \phi_L\}. \tag{9}$$

7.2 Source Photometric Model: Elliptical Sérsic Profile

The background source surface-brightness distribution is parameterised by an elliptical Sérsic profile. The Sérsic model is an empirical photometric function capable of representing a wide range of galaxy morphologies — from exponential disc profiles ($n = 1$) to de Vaucouleurs elliptical profiles ($n = 4$) — with a single shape parameter, making it well-suited to the front-end rapid modelling stage.

Radial profile

The isotropic Sérsic surface-brightness profile is

$$I(R) = I_e \exp\left\{-b_n \left[\left(\frac{R}{R_e}\right)^{1/n} - 1\right]\right\}, \tag{10}$$

where I_e is the surface brightness at the effective (half-light) radius R_e, n is the Sérsic index, and b_n is a normalisation constant determined by the implicit condition

$$\Gamma(2n) = 2\,\gamma(2n,\, b_n), \tag{11}$$

with Γ and γ denoting the complete and lower incomplete gamma functions, respectively. For practical computation, the high-accuracy asymptotic approximation

$$b_n \approx 2n - \frac{1}{3} + \frac{4}{405n} + \mathcal{O}(n^{-2}) \tag{12}$$

is used, which is accurate to better than 0.1% for $0.5 \lesssim n \lesssim 10$.

Elliptical extension

To capture non-circular morphologies, the profile is generalised by defining the elliptical radius

$$R = \sqrt{x'^2 + \frac{y'^2}{q_S^2}}, \tag{13}$$

where $q_S \in (0, 1]$ is the source-plane axis ratio and (x', y') are coordinates in the source principal-axis frame:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\phi_S & \sin\phi_S \\ -\sin\phi_S & \cos\phi_S \end{pmatrix} \begin{pmatrix} x - x_S \\ y - y_S \end{pmatrix}, \tag{14}$$

with (x_S, y_S) the source centroid and ϕ_S the source position angle.

Parameter set

The elliptical Sérsic source model is fully characterised by the seven-parameter vector

$$\boldsymbol{\pi}_S = \{I_e,\ R_e,\ n,\ x_S,\ y_S,\ q_S,\ \phi_S\}. \tag{15}$$

This parameterisation provides a favourable trade-off between the number of free parameters and the range of representable morphologies, and is particularly suited to front-end parameter-space exploration.

Scope and limitations. Real background source galaxies may exhibit substructure, multi-component morphology, or significant non-elliptical asymmetry that cannot be captured by a single Sérsic profile. Within the Hoopla modelling pipeline, the primary function of the parametric source model is to supply a physically consistent and stable geometric constraint on the lens mass model, rather than to precisely reconstruct the source morphology. Residual source complexity is best addressed in the back-end fine-modelling stage through pixelated source reconstruction or multi-component parametric models.

8 Frequently Asked Questions

Q1: The χ^2 curve shows significant jumps or oscillations during optimisation — is this expected? Moderate fluctuations in χ^2 during the early iterations are a normal feature of the Nelder–Mead algorithm, which performs global exploration of the parameter space by periodically escaping local optima. If large-amplitude oscillations or the absence of a clear downward trend persist in the late optimisation phase, the configuration has likely not converged. In this situation, the residual map should be inspected for the presence of residual physical structure, which indicates that the model is not adequately describing the data.

Q2: Model parameters drift toward physically extreme values (*e.g.* anomalously large axis ratio or Einstein radius) — does this indicate model failure? A drift toward physically extreme parameter values is a diagnostic signal indicating that the optimisation has become trapped in a local minimum, or that the model assumptions are inconsistent with the

data. Potential causes include inadequate initial parameter estimates, an insufficient number of optimisation iterations, systematic errors in the weight map, or insufficient constraints on the parameter space from the data. It is recommended to suspend the optimisation, reinspect and revise the initial model configuration, and refrain from using such results for scientific analysis. Back-end MCMC sampling over a broader parameter range should be used to characterise the parameter space systematically.

Q3: χ^2 has decreased substantially, but the image-plane reconstruction quality is still poor — how should this be handled? A decreasing χ^2 value does not guarantee physically meaningful model convergence, particularly when the weight map is miscalibrated or the initial model is far from the true solution: the optimiser may minimise residuals in low-weight regions at the expense of the scientifically relevant arc structure. Under such circumstances, the spatial position, number, and overall morphology of the predicted arcs in the Image Plane canvas should be used as the primary qualitative criterion. Adjusting the mask region, revising the weight map, or supplying a more physically motivated initial model configuration are the recommended remediation steps.

Q4: After invoking `Extract Lens Arc`, residual background contamination remains in the extracted image — is this expected, and how should it be handled? For a small fraction of images with low signal-to-noise ratio, indistinct arc structure, or multiple background sources, GGSL-UNet may produce segmentations in which arc and contaminant flux coexist. In such cases, the user may set an appropriate mask radius via `Apply Mask` to exclude the contaminating region, or alternatively draw a mask on the original image before invoking arc extraction.

9 Limitations and Usage Recommendations

9.1 Model Assumption Constraints

The current implementation of Hoopla is restricted to galaxy-scale strong gravitational lensing with a single-lens, single-source configuration. The adopted SIE mass model and Sérsic source model are not appropriate for galaxy-cluster lensing, multi-plane lensing, or systems requiring multi-component or pixelated mass reconstructions.

9.2 Arc-Extraction Performance Limitations

When processing real observational data, GGSL-UNet is invoked to suppress foreground light and isolate the lensed arc structure. For gravitational lens systems with low arc signal-to-noise ratio or poorly resolved arc morphology, segmentation performance may degrade substantially, yielding an arc image that lacks the geometric clarity required to constrain the lens model. In such cases, the user is advised to conduct manual interactive modelling against the original observed image, treating any automated segmentation output as auxiliary information. It should be noted that in the absence of a reliable arc extraction result, the front-end optimiser lacks sufficient spatial constraints to produce well-defined parameter estimates.

9.3 Limitations of Front-End Parameter Estimates

The front-end optimisation module employs the Nelder–Mead algorithm, which is designed to minimise computation time and support interactive feedback rather than to achieve rigorous statistical inference. The parameter estimates produced at the front-end stage should therefore be interpreted as exploratory point estimates that define a plausible region of the parameter space, and not as maximum-likelihood or posterior estimates with well-defined statistical uncertainties.

Statistically rigorous parameter constraints and high-fidelity lens reconstruction require the back-end sampling methods (*e.g.* MCMC or nested sampling) provided through TinyLensGPU.

9.4 Operational Recommendations

(1) Treat Hoopla as the interactive front-end layer of a complete lensing pipeline rather than as a self-contained modelling tool. Its primary function is to deliver physically motivated, high-confidence prior estimates for back-end inference.

(2) During interactive modelling, prioritise structural agreement between the predicted and observed arc morphology — arc position, arc count, and overall geometry — over pixel-level residual minimisation. Small residuals at sub-arc-second scales are more appropriately addressed during back-end fine-modelling.

(3) Given the inherent parameter degeneracies in gravitational lens modelling, explore multiple initial model configurations and compare the corresponding image-plane reconstructions before committing to a specific parameter set. Saving and comparing multiple model files facilitates assessment of sensitivity to initial conditions and provides a range of priors for the back-end sampler.

(4) Evaluate the applicability of the automated arc extraction module on a per-system basis. For data with marginal quality, manual interactive modelling against the original image is preferable; automated segmentation results should be treated as supplementary rather than authoritative.

(5) Hoopla's modular design explicitly decouples the front-end interaction layer from the image-segmentation and back-end inference modules. Alternative segmentation algorithms or back-end samplers may be integrated by replacing the respective module without modifying the front-end codebase, enabling Hoopla to serve as a unified interactive entry point for diverse strong-lensing research workflows.

(6) To ensure reproducibility and traceability of scientific results, retain the complete set of modelling artefacts: the exported model `.json` file, the lens image, the weight map, and the PSF description. In any subsequent analysis or publication, clearly distinguish between front-end interactive modelling outputs and back-end statistically inferred results.

References

References

[1] Cao, X. et al. (2025). *TinyLensGPU: A JAX-accelerated framework for galaxy-scale strong gravitational lensing modelling.* MNRAS, 540, 3121. https://github.com/caoxiaoyue/TinyLensGpu

[2] Zhong, F. et al. *GGSL-UNet: A deep learning model for simultaneous extraction of foreground lens and background arc photometry in galaxy-scale strong gravitational lensing.* https://github.com/Fucheng-Zhong/GGSL-UNet